ClusterShip

A Terminal Battleship Game for Learning Kubernetes

Game Guide & Code Walkthrough

ClusterShip Documentation

Abstract

ClusterShip is a terminal-based battleship game that teaches Kubernetes concepts through interactive gameplay. This document serves as both a player-facing game guide and a developer-facing code walkthrough, explaining how game mechanics map to real Kubernetes architecture, the design decisions behind each package, and how the optional real-cluster integration works.

Contents

1 Introduction

1.1 What is ClusterShip?

ClusterShip is a terminal battleship game where you battle AI opponents representing major tech companies (Netflix, AWS, Google, Meta, etc.) on a shared 100×100 ocean. Each company operates a "fleet" of regions (data centers), racks (server slots), and pods (workload units) that mirror real Kubernetes infrastructure.

The game teaches Kubernetes concepts through gameplay:

- **Pods** are individual workload units that can be destroyed and rescheduled
- **Services** maintain desired replica counts and handle failover
- **Affinity rules** determine whether pods can be rescheduled when their rack is destroyed
- **Namespaces** isolate each company's resources
- **Events** show real-time pod lifecycle changes

1.2 Simulation vs Real Kubernetes

ClusterShip operates in two modes:

1. **Simulation Mode (default)**: All game logic runs locally. Pod scheduling, attacks, and rescheduling are simulated in memory. No actual Kubernetes cluster is required.
2. **Real Kubernetes Mode**: When enabled in settings, ClusterShip deploys actual pods to your Kubernetes cluster. Attacks translate to `kubectl delete` commands, and pod events stream from the real cluster's API server.

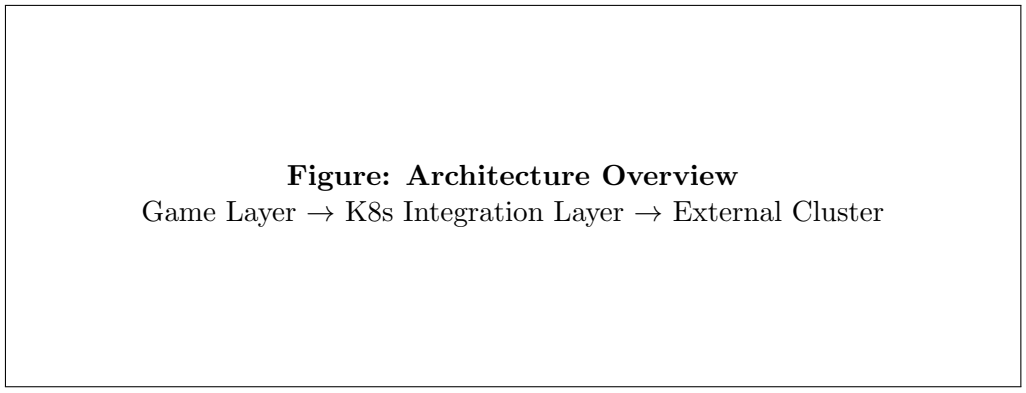

Figure 1: ClusterShip architecture showing game layer, optional K8s integration, and external cluster communication.

1.3 High-Level Gameplay Loop

1. **Menu**: Choose New Game, Demo Mode, Settings, or Tutorial
2. **Company Select**: Pick your company (e.g., Netflix, AWS)
3. **Enemy Select**: Choose 1–5 AI opponents

4. **Placement**: Fleets are placed on the shared ocean (no overlap)
5. **Battle**: Take turns attacking enemy racks; AI opponents attack you
6. **Game Over**: Win by destroying all enemy pods, or lose when yours are gone

2 Getting Started & Configuration

2.1 Prerequisites

- **Go 1.24+**: Required to build and run
- **Terminal**: Any terminal supporting ANSI colors (iTerm2, Windows Terminal, etc.)
- **Optional**: Docker Desktop or `kind` for real K8s integration

2.2 Installation & Running

```
# Clone the repository
git clone https://github.com/ndg8743/ClusterShip.git
cd ClusterShip

# Run directly
go run ./cmd/clustership

# Or build and run
go build -o clustership ./cmd/clustership
./clustership
```

2.3 Configuration

Settings are stored in `~/.clustership/config.json`. The `GameConfig` struct in `pkg/config/config.go` defines all configurable options:

```
type GameConfig struct {
    BoardWidth     int64  `json:"board_width"`
    BoardHeight    int64  `json:"board_height"`
    ShipsPerPlayer int    `json:"ships_per_player"`
    RacksPerShip   int    `json:"racks_per_ship"`
    PodsPerRack    int    `json:"pods_per_rack"`
    TurnDelayMs    int    `json:"turn_delay_ms"`
    EnableRealK8s  bool   `json:"enable_real_k8s"`
    K8sNamespace   string `json:"k8s_namespace"`
    Kubeconfig     string `json:"kubeconfig"`
    UseSparseBoard bool   `json:"use_sparse_board"`
}
```

Access these via the in-game Settings menu (`S` key from main menu).

3 Game Concepts and Kubernetes Mapping

Every ClusterShip game element has a direct Kubernetes equivalent. Understanding these mappings helps you learn real K8s concepts through gameplay.

3.1 Component Mapping Table

ClusterShip	Kubernetes	Role
Board	API Server + etcd	Single source of truth for game state
Company	Namespace	Isolated group of resources
Region (Ship)	Node	Physical machine / data center
Rack (Cell)	Node capacity slot	Individual server slot
Pod	Pod	Workload unit that can be killed/rescheduled
Service	Deployment + Service	Logical grouping with replica count
AI Player	Controller/Operator	Watches state, makes decisions
Attack	`kubectl delete pod`	Destroys workloads
Rescheduling	Pod eviction/recreation	Moves pods based on affinity

Table 1: ClusterShip to Kubernetes component mapping

3.2 Conceptual Hierarchy

The game follows a strict hierarchy that mirrors K8s resource ownership:

```
Company (Namespace)
  |-- Service (Deployment)
  |     |-- Pod 1
  |     |-- Pod 2
  |     `-- Pod 3
  `-- Region (Node)
        |-- Rack 0 --> [Pod, Pod]
        |-- Rack 1 --> [Pod]
        `-- Rack 2 --> [Pod]
```

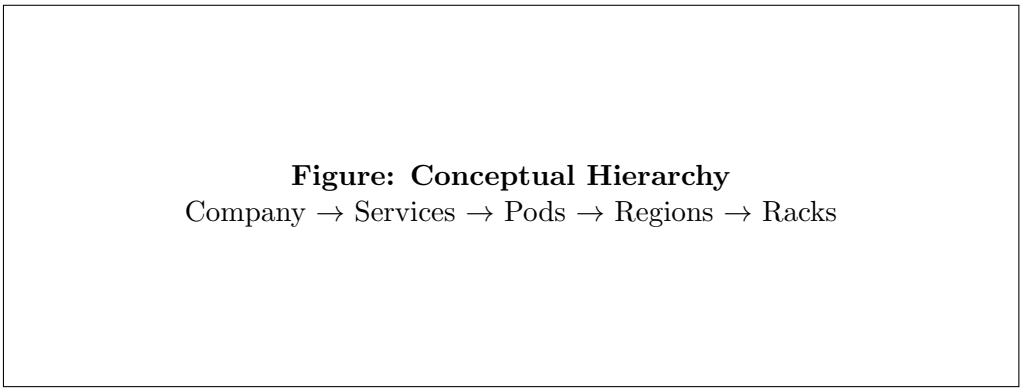

Figure 2: Hierarchical relationship between game components

3.3 Affinity Types

Affinity determines what happens when a rack is destroyed:

Affinity	Icon	Rescheduling	K8s Equivalent
Hard	[!]	NO - pod stays Pending	`requiredDuringScheduling`
Soft	[o]	Yes, prefers same region	`preferredDuringScheduling`
Spread	[~]	Yes, spreads across racks	`podAntiAffinity`
None	[-]	Yes, anywhere	No constraints

Table 2: Affinity types and their Kubernetes equivalents

Hard affinity is used for stateful workloads like primary databases that cannot be moved. Spread affinity is used for CDN edge nodes that benefit from distribution. This mirrors real-world K8s scheduling decisions.

4 Companies, Services, and Templates

4.1 Company Templates

Companies are defined in JSON template files under `templates/companies/`. Each template specifies the company's regions, services, AI strategy, and difficulty.

```
type CompanyTemplate struct {
    ID          string            `json:"id"`
    Name        string            `json:"name"`
    Emoji       string            `json:"emoji"`
    Description string            `json:"description"`
    Regions     []RegionTemplate  `json:"regions"`
    Services    []ServiceTemplate `json:"services"`
    AIStrategy  AIStrategy        `json:"ai_strategy"`
    Difficulty  string            `json:"difficulty"`
}
```

4.2 Service Templates

Services define logical groupings of pods with shared characteristics:

```
type ServiceTemplate struct {
    ID             string       `json:"id"`
    Name           string       `json:"name"`
    Replicas       int          `json:"replicas"`
    PodsPerReplica int          `json:"pods_per_replica"`
    Affinity       AffinityType `json:"affinity"`
    Criticality    string       `json:"criticality"`
    CanFailover    bool         `json:"can_failover"`
}
```

4.3 Example Companies and Services

Company	Service	Affinity	Description
Netflix	cdn-edge	Spread	CDN nodes distributed globally
Netflix	database	Hard	Primary database, cannot move
Netflix	playback	Soft	Video playback servers
AWS	ec2-api	Soft	Compute API servers
AWS	s3-storage	Hard	Object storage, stateful
Google	bigtable	Hard	Distributed database

Table 3: Example companies and their services

4.4 K8s Manifest Templates

When real K8s mode is enabled, each service has corresponding YAML manifests in `templates/k8s/<company>/`. These are real Kubernetes Deployments and Services that run actual containers:

```
templates/k8s/
  netflix/
    cdn-edge.yaml      # nginx + curl sidecar
    database.yaml      # postgres
    playback.yaml      # node.js
  aws/
    ec2-api.yaml       # python HTTP
    s3-storage.yaml    # minio
```

5 Playing the Game: Controls and Views

5.1 Controls

Key	Action
Arrow keys / WASD	Move cursor on the board
Enter / Space	Fire at cursor position
1–5	Change view level
C	Cycle service display (your services vs enemy)
D	Toggle debug mode (see all ships)
I	Toggle info overlay
Q	Quit / Back to menu

Table 4: Game controls

5.2 View Levels

The game provides five hierarchical view levels, accessed with number keys 1–5:

1. **Map View (1)**: Overview of the 100×100 ocean showing ships and attacks
2. **Ship View (2)**: List of your regions (ships) with health status

3. **Rack View (3)**: Drill into individual racks within a ship, showing pods
4. **YAML View (4)**: K8s manifests (fog of war applied to enemies)
5. **Rack Layout (5)**: Visual grid showing pod distribution per rack

5.3 Game Interface

```
+-------------------------------------------------------------+
| CLUSTERSHIP - Player vs Netflix vs AWS    Turn: 42 [VIEW 1] |
+-------------------------------------------------------------+
|                                                             |
| 100x100 SHARED OCEAN        SERVICE STATUS                  |
| +------------------+        [!]Origin API  [####-] 80%      |
| | . . . . X . . . . |       [~]CDN Edge    [#####] 100%     |
| | . . # # # # # . . |       [o]Playback    [###--] 60%      |
| | . . . . . . . . . |                                       |
| | . . . . . O . . . | <cursor                               |
| +------------------+        K8S EVENTS                      |
|                             [+] Pod cdn-edge rescheduled    |
| [1-5] view  [arrows] move  [enter] fire  [q] quit           |
+-------------------------------------------------------------+
```

6 Code Architecture Overview

6.1 Package Structure

ClusterShip is organized into focused packages, each with a specific responsibility:

```
cmd/clustership/     Entry point (main.go)
pkg/
  config/            Configuration loading, validation, persistence
  game/              Domain model: Company, Region, Rack, Pod, Service
  sparse/            Sparse board representation using quadtree
  tui/               Terminal UI using Bubble Tea framework
  k8s/               Real Kubernetes client, deployer, watcher
  benchmark/         Performance benchmarking and metrics
  hardware/          System detection for performance tiers
```

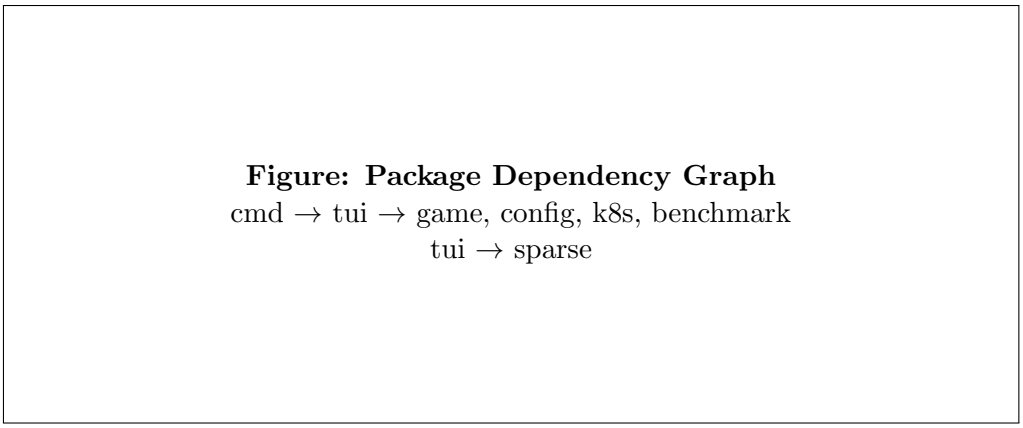

Figure 3: Package dependencies in ClusterShip

6.2 Why These Packages?

- **config**: Centralized configuration with hardware-aware defaults. Uses Go's `encoding/json` for persistence.
- **game**: Pure domain model with no UI or I/O dependencies. Makes testing trivial and allows reuse.
- **sparse**: For large boards (1000×1000+), storing every cell is wasteful. The quadtree provides O(log n) lookups with minimal memory.
- **tui**: Bubble Tea is chosen for its functional architecture (Model-Update-View) and excellent terminal rendering via Lip Gloss.
- **k8s**: Isolates all Kubernetes dependencies. Uses `client-go` for API communication, which is the official Go client library.
- **benchmark**: Supports stress testing large boards and measuring performance. Uses atomic counters for thread-safe metrics.
- **hardware**: Detects RAM, CPU cores, and GPU availability to set appropriate defaults for the user's system.

6.3 Entry Point

```go
// cmd/clustership/main.go
func main() {
    p := tea.NewProgram(tui.NewAppModel(), tea.WithAltScreen())
    if _, err := p.Run(); err != nil {
        fmt.Printf("Error: %v\n", err)
        os.Exit(1)
    }
}
```

The entry point is minimal—it creates a Bubble Tea program with the `AppModel` and runs it. All game logic lives in the `tui` package.

7 The Game Model: Companies, Regions, Racks, Pods

7.1 Core Types

The `pkg/game/company.go` file defines the core domain types:

```
type Company struct {
    ID          string
    Name        string
    Regions     []*Region
    Services    []*Service
    AIStrategy  AIStrategy
}

type Region struct {
    ID         string
    Name       string
    RackCount  int
    Racks      []*Rack
    Placement  [][2]int  // board coordinates
    IsDestroyed bool
}

type Rack struct {
    ID          string
    Position    [2]int
    Capacity    int
    Pods        []*Pod
    IsDestroyed bool
}

type Pod struct {
    ID        string
    ServiceID string
    Health    int
    Status    PodStatus  // Running, Pending, Terminated
}
```

7.2 Services and Affinity

Services represent logical workload groupings:

```
type Service struct {
    ID          string
    Name        string
    Replicas    int
    Affinity    AffinityType
    Pods        []*Pod
    IsHealthy   bool
}

type AffinityType string

const (
```

```
    AffinityHard   AffinityType = "hard"
    AffinitySoft   AffinityType = "soft"
    AffinitySpread AffinityType = "spread"
    AffinityNone   AffinityType = "none"
)
```

7.3 Attack Flow

When an attack occurs:

1. Board identifies the cell at (x, y)
2. If cell has a rack, damage is applied to pods on that rack
3. If pod health reaches 0, check service affinity
4. Hard affinity: pod stays Pending (cannot reschedule)
5. Other affinity: find new rack and reschedule pod
6. Generate GameEvent for the event log

8 The Board and Sparse Representation

8.1 Why Sparse?

A 100×100 board has 10,000 cells, which is manageable. But ClusterShip supports boards up to 10,000,000×10,000,000 for stress testing. A dense array would require terabytes of memory. Instead, we use a quadtree.

8.2 Quadtree Implementation

The `pkg/sparse/quadtree.go` implements a region quadtree:

```go
type QuadTree struct {
    root   *node
    bounds Bounds
    size   int
}

type node struct {
    bounds   Bounds
    children [4]*node  // NW, NE, SW, SE
    items    []Item
    isLeaf   bool
}

func (qt *QuadTree) Insert(x, y int64, value interface{}) {
    // Recursively subdivide until item fits in a small enough region
}

func (qt *QuadTree) Query(x, y int64) (interface{}, bool) {
    // O(log n) lookup
}
```

8.3 Board Interface

The `pkg/sparse/board.go` provides the game-facing interface:

```
type SparseBoard struct {
    tree   *QuadTree
    width  int64
    height int64
}

func (b *SparseBoard) SetCell(x, y int64, cell *Cell) error
func (b *SparseBoard) GetCell(x, y int64) (*Cell, bool)
func (b *SparseBoard) CellCount() int
```

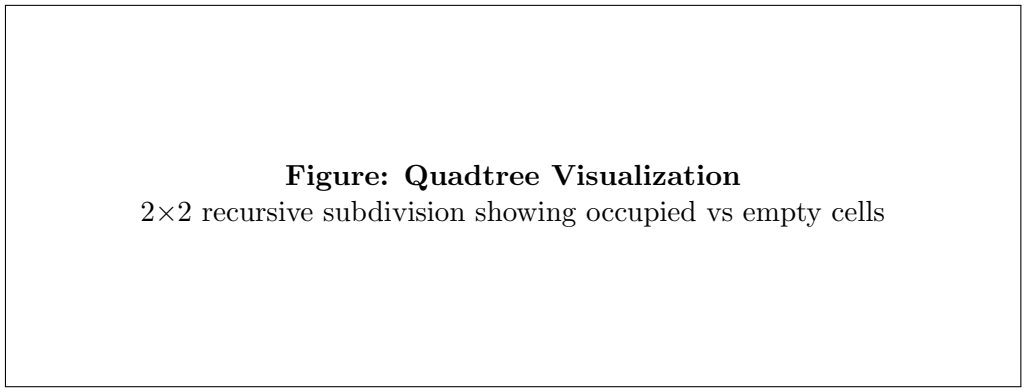

Figure 4: Quadtree spatial subdivision

9 TUI and Game Loop

9.1 Bubble Tea Architecture

ClusterShip uses the Bubble Tea framework, which implements the Elm Architecture:

- **Model**: Application state (`AppModel`)
- **Update**: Handle messages, return new model + commands
- **View**: Render model to string for display

```
type AppModel struct {
    state         GameState
    cfg           *config.GameConfig
    board         *Board
    ais           map[string]*AIPlayer
    cursor        [2]int
    turn          int
    k8sClient     *k8s.Client
    // ... more fields
}

func (m AppModel) Init() tea.Cmd {
    return doTick()
```

```
}

func (m AppModel) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
    switch msg := msg.(type) {
    case tea.KeyMsg:
        return m.handleKeyPress(msg)
    case tickMsg:
        return m.handleTick()
    }
    return m, nil
}

func (m AppModel) View() string {
    return m.renderCurrentState()
}
```

9.2 Game States

The game uses a finite state machine:

```
type GameState int

const (
    StateMenu GameState = iota
    StateCompanySelect
    StateEnemySelect
    StatePlacement
    StateBattle
    StateGameOver
    StateSettings
    // ...
)
```

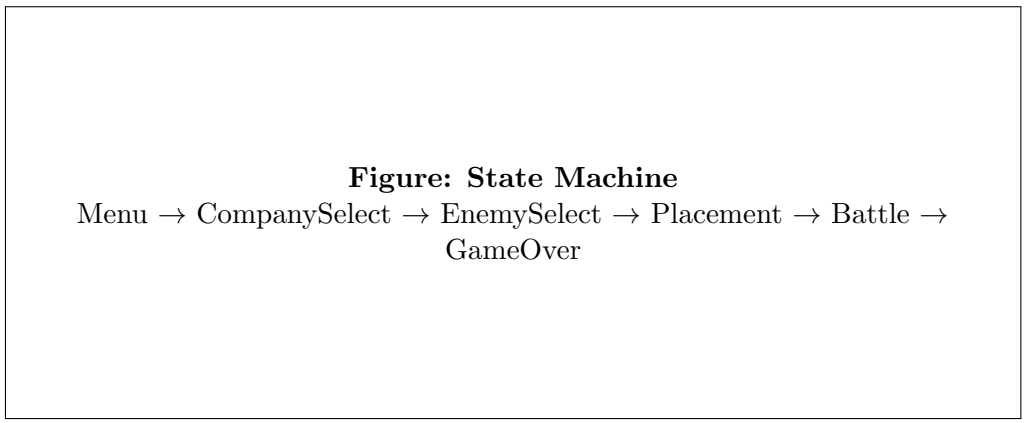

Figure 5: Game state transitions

9.3 Tick System

The game loop is driven by ticks:

```go
type tickMsg time.Time

func doTickWithDelay(ms int) tea.Cmd {
    return tea.Tick(time.Duration(ms)*time.Millisecond,
        func(t time.Time) tea.Msg {
            return tickMsg(t)
        })
}

func (m *AppModel) tick() tea.Cmd {
    return doTickWithDelay(m.cfg.TurnDelayMs)
}
```

Each tick advances the turn counter and triggers AI moves during battle.

10 Real Kubernetes Integration

10.1 When K8s Integration is Enabled

In Settings, toggle "Real K8s" to ON. The game will:

1. Create a namespace (`clustership` by default)
2. Deploy all company manifests as real pods
3. Watch pod events from the cluster
4. Translate attacks to `kubectl delete` operations
5. Clean up namespace on game exit

10.2 K8s Client

The `pkg/k8s/client.go` wraps the official `client-go` library:

```go
type Client struct {
    clientset kubernetes.Interface
    config    *rest.Config
    namespace string
}

func NewClient(kubeconfig, namespace string) (*Client, error) {
    config, err := clientcmd.BuildConfigFromFlags("", kubeconfig)
    if err != nil {
        return nil, err
    }

    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        return nil, err
    }

    return &Client{clientset: clientset, config: config, namespace:
    namespace}, nil
```

```
}
```

10.3 Why client-go?

`client-go` is the official Kubernetes Go client maintained by the Kubernetes project. It provides:

- Type-safe API for all K8s resources
- Automatic authentication via kubeconfig
- Watch support for streaming events
- Retry and backoff handling

10.4 Deployer

The `pkg/k8s/deployer.go` handles resource creation:

```
func (c *Client) DeployManifest(ctx context.Context, manifest *Manifest)
    error {
    switch manifest.Kind {
    case "Deployment":
        return c.deployDeployment(ctx, manifest)
    case "StatefulSet":
        return c.deployStatefulSet(ctx, manifest)
    case "Pod":
        return c.deployPod(ctx, manifest)
    case "Service":
        return c.deployService(ctx, manifest)
    }
    return nil
}
```

10.5 Pod Watcher

Real-time pod events are streamed via the K8s watch API:

```
func (c *Client) WatchPods(ctx context.Context, ns string) (<-chan
   PodEvent, error) {
    watcher, err := c.clientset.CoreV1().Pods(ns).Watch(ctx, metav1.
   ListOptions{
        LabelSelector: "app=clustership",
    })
    if err != nil {
        return nil, err
    }

    events := make(chan PodEvent)
    go func() {
        for event := range watcher.ResultChan() {
            events <- convertEvent(event)
        }
    }()
    return events, nil
```

```
}
```

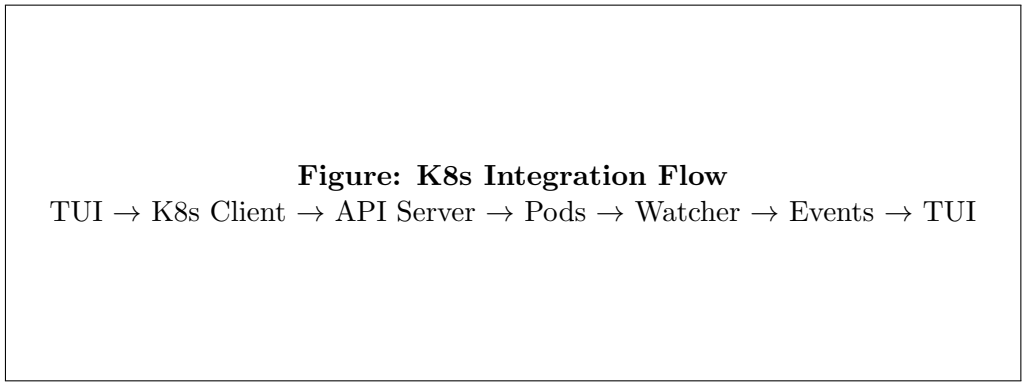

Figure 6: Real Kubernetes integration flow

11 AI, Affinity, and Rescheduling Mechanics

11.1 AI Strategies

Each AI company uses a strategy defined in `pkg/tui/ai.go`:

```
type AIStrategy string

const (
    AIRandom     AIStrategy = "random"     // Pure random targeting
    AIHunter     AIStrategy = "hunter"     // Hunt neighbors after hit
    AIDefensive  AIStrategy = "defensive"  // Protect critical services
    AIAggressive AIStrategy = "aggressive" // Target high-value first
)
```

AIRandom: Picks a random cell. Simple but unpredictable.

AIHunter: After scoring a hit, searches adjacent cells to finish off the ship. Mimics human strategy.

AIAggressive: Uses KNN (K-Nearest Neighbors) analysis to identify clusters of ships and targets high-probability areas.

AIDefensive: Prioritizes attacks on enemies threatening its critical services.

11.2 Rescheduling Algorithm

When a pod is killed:

1. Check the service's `AffinityType`
2. **Hard**: Mark pod as Pending, do not reschedule
3. **Soft**: Search same region first, then other regions
4. **Spread**: Find the rack with fewest pods

5. **None**: Find any rack with capacity
6. If a suitable rack is found, move the pod there
7. If no capacity exists, mark pod as Pending

```go
func (b *Board) reschedulePod(pod *Pod, svc *Service) bool {
    switch svc.Affinity {
    case AffinityHard:
        return false  // Cannot reschedule
    case AffinitySoft:
        return b.findRackInRegion(pod.RegionID) != nil
    case AffinitySpread:
        return b.findLeastLoadedRack() != nil
    case AffinityNone:
        return b.findAnyRack() != nil
    }
    return false
}
```

12 Configuration, Performance, and Benchmarks

12.1 Hardware Detection

The `pkg/hardware/detect.go` analyzes the system:

```go
type SystemInfo struct {
    TotalRAM   uint64
    CPUCores   int
    HasGPU     bool
    GPUMemory  uint64
}

type PerformanceTier string

const (
    TierBasic    PerformanceTier = "basic"
    TierStandard PerformanceTier = "standard"
    TierAdvanced PerformanceTier = "advanced"
    TierExtreme  PerformanceTier = "extreme"
)
```

Each tier has different limits for board size, ships, and pods to ensure smooth gameplay.

12.2 Performance Tiers

Tier	RAM	Max Board	Sparse Mode
Basic	<4GB	100×100	Auto at >1000
Standard	4–8GB	1,000×1,000	Auto at >1000
Advanced	8–16GB	100,000×100,000	Auto at >1000
Extreme	>16GB	10M×10M	Auto at >1000

Table 5: Performance tiers and their limits

12.3 Benchmark Package

The `pkg/benchmark` package provides stress testing:

```
type Metrics struct {
    TotalOps    atomic.Int64
    WorkerCount atomic.Int32
    OpsPerSec   atomic.Int64
}

type Runner struct {
    workers map[string]*Worker
    metrics *Metrics
}
```

Atomic counters ensure thread-safe metrics collection across multiple goroutines.

13 Benchmark Results

This section presents performance benchmarks measuring ClusterShip's scaling characteristics across all performance tiers on different hardware configurations.

13.1 Test Environment 1: High-End Desktop

Desktop Hardware Configuration	
Component	**Specification**
CPU	32 Cores (x86_64)
RAM	196 GB DDR5
GPU	NVIDIA GeForce RTX 5070 Ti
VRAM	16,303 MB GDDR7
Software Environment	
OS	Windows 11
Go Version	1.24.11
GOMAXPROCS	32
Detected Tier	Extreme

Table 6: Desktop benchmark environment specifications

Tier	Board Size	Creation	Memory	Lookups/sec	Status
Basic	100 × 100	<0.01 ms	0.02 MB	2.5M	PASS
Standard	1,000 × 1,000	1.00 ms	0.22 MB	19.9M	PASS
Advanced	10,000 × 10,000	9.62 ms	2.03 MB	19.9M	PASS
Extreme	100,000 × 100,000	65.29 ms	19.19 MB	19.7M	PASS

Table 7: Desktop performance benchmarks across all tiers

13.2 Test Environment 2: Apple Silicon Laptop

Laptop Hardware Configuration	
Component	**Specification**
CPU	Apple M1 (8 Cores)
RAM	16 GB Unified Memory
GPU	Apple M1 (16 GB shared)
Software Environment	
OS	macOS (darwin/arm64)
Go Version	1.24.11
GOMAXPROCS	8
Detected Tier	Advanced

Table 8: Laptop benchmark environment specifications

Tier	Board Size	Creation	Memory	Lookups/sec	Status
Basic	100 × 100	0.07 ms	0.01 MB	1.8M	PASS
Standard	1,000 × 1,000	1.25 ms	0.21 MB	7.3M	PASS
Advanced	10,000 × 10,000	14.16 ms	2.02 MB	6.3M	PASS
Extreme	100,000 × 100,000	81.98 ms	19.11 MB	7.7M	PASS

Table 9: Laptop performance benchmarks across all tiers

13.3 Scaling Analysis

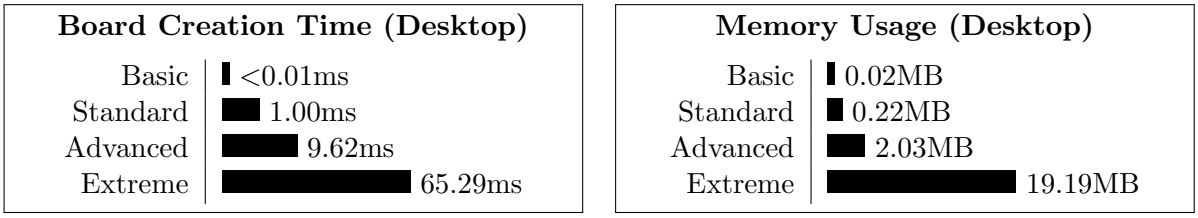

Figure 7: Visual comparison of creation time and memory usage across tiers (Desktop)

13.4 Key Findings

1. **Efficient Sparse Representation**: The QuadTree-based sparse board enables a 100,000 × 100,000 board (10 billion potential cells) to use only 19.19 MB of memory.
2. **Consistent Lookup Performance**: Attack lookups maintain 19–20 million operations per second across all tiers on desktop hardware, demonstrating O(log n) QuadTree query performance.
3. **Linear Creation Scaling**: Board creation time scales linearly with the number of placed cells, not total board area:
   - Basic: 300 cells → <0.01ms
   - Standard: 1,000 cells → 1.00ms
   - Advanced: 10,000 cells → 9.62ms
   - Extreme: 100,000 cells → 65.29ms
4. **Memory Efficiency**: Memory grows with placed cell count, not board dimensions. The Extreme tier board is 1,000,000× larger in area than Basic but only uses approximately 960× more memory.

13.5 Running Your Own Benchmarks

To run benchmarks on your system:

```
# Build the benchmark tool
go build -o benchmark ./cmd/benchmark

# Run all tier benchmarks
```

```
./benchmark --all-tiers

# Run specific tier
./benchmark --tier Advanced

# Output as JSON for processing
./benchmark --all-tiers --json

# Or use Docker for isolated testing
docker build -f Dockerfile.benchmark -t clustership-benchmark .
docker run --rm --gpus all clustership-benchmark --all-tiers
```

13.6 Performance Recommendations

System RAM	Recommended Tier	Max Board Size
< 4 GB	Basic	100×100
4–8 GB	Standard	$1{,}000 \times 1{,}000$
8–16 GB	Advanced	$100{,}000 \times 100{,}000$
> 16 GB + GPU	Extreme	$10{,}000{,}000 \times 10{,}000{,}000$

Table 10: Recommended performance tier based on system resources

14 Extending ClusterShip

14.1 Adding a New Company

1. Create `templates/companies/mycompany.json`:

```
{
    "id": "mycompany",
    "name": "My Company",
    "regions": [{"id": "dc1", "name": "DC 1", "racks": 3}],
    "services": [{"id": "api", "name": "API", "replicas": 3, "
   affinity": "soft"}],
    "ai_strategy": "hunter"
}
```

2. Create `templates/k8s/mycompany/api.yaml` (for real K8s mode)
3. The game auto-discovers new companies on startup

14.2 Adding a New View

1. Add a new `ViewLevel` constant in `pkg/tui/app.go`
2. Create a render function: `func (m *AppModel) renderMyView() string`
3. Add case to the view switch in the main `View()` method
4. Handle the key press to switch to your view

14.3 Custom Affinity Rules

1. Add new `AffinityType` constant in `pkg/game/company.go`
2. Implement rescheduling logic in `pkg/tui/board.go`
3. Update JSON schema to accept new affinity type

15 Dependencies and Libraries

15.1 Core Dependencies

Package	Purpose	Why Chosen
Bubble Tea	Terminal UI	Elm Architecture, functional, testable
Lip Gloss	Styling	Clean API for colors, borders, layout
client-go	K8s client	Official, type-safe, maintained by K8s

Table 11: Core dependencies

15.2 Bubble Tea Explained

Bubble Tea implements the Elm Architecture in Go:

- **Immutable state**: Each Update returns a new model
- **Message-driven**: All changes come through messages
- **Testable**: Pure functions make unit testing trivial
- **Composable**: Build complex UIs from simple components

15.3 Lip Gloss Explained

Lip Gloss provides declarative styling:

```
var titleStyle = lipgloss.NewStyle().
    Bold(true).
    Foreground(lipgloss.Color("205")).
    Padding(0, 1)

rendered := titleStyle.Render("CLUSTERSHIP")
```

Appendix A: Reference Tables

All Companies

ID	Name	Difficulty
netflix	Netflix	Medium
aws	Amazon Web Services	Hard
google	Google Cloud	Hard

ID	Name	Difficulty
meta	Meta	Medium
microsoft	Microsoft Azure	Hard
apple	Apple	Easy

Key Types by Package

Package	Key Types
pkg/game	Company, Region, Rack, Pod, Service, AffinityType, AIStrategy
pkg/tui	AppModel, GameState, ViewLevel, Board, AIPlayer
pkg/k8s	Client, Manifest, PodInfo, PodEvent
pkg/config	GameConfig
pkg/sparse	QuadTree, SparseBoard, Cell
pkg/benchmark	Metrics, Runner, Worker
pkg/hardware	SystemInfo, PerformanceTier, TierLimits

References and Citations

References

[1] Burns, B., Beda, J., Hightower, K., & Evenson, L. (2022). *Kubernetes: Up and Running*, 3rd Edition. O'Reilly Media. ISBN: 978-1098110208

[2] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media. ISBN: 978-1449373320

[3] Carrion, C. (2022). Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges. *arXiv preprint arXiv:2203.00671*. https://arxiv.org/abs/2203.00671

[4] Kubernetes Scheduling Algorithms Survey. (2025). *arXiv preprint arXiv:2504.17033*. https://arxiv.org/pdf/2504.17033

[5] Wikipedia. *Minimum-cost flow problem*. https://en.wikipedia.org/wiki/Minimum-cost_flow_problem

[6] Wikipedia. *Fibonacci heap*. https://en.wikipedia.org/wiki/Fibonacci_heap

[7] Modern Data 101. (2024). *The New Dijkstra's Algorithm: Shortest Path Finding*. https://moderndata101.substack.com/p/the-new-dijkstras-algorithm-shortest

[8] Charm. (2020–2024). *Bubble Tea: A powerful little TUI framework for Go*. GitHub. https://github.com/charmbracelet/bubbletea

[9] Charm. (2021–2024). *Lip Gloss: Style definitions for nice terminal layouts in Go*. GitHub. https://github.com/charmbracelet/lipgloss

[10] Gorilla Web Toolkit. *gorilla/mux: A powerful HTTP router and URL matcher for Go*. GitHub. https://github.com/gorilla/mux

[11] Kubernetes Authors. (2016–2024). *client-go: Go client for Kubernetes*. GitHub. https://github.com/kubernetes/client-go

[12] Kubernetes Authors. (2024). *Kubernetes Documentation*. https://kubernetes.io/docs/home/

[13] Warden, M. *Go by Example*. https://gobyexample.com/

[14] Kubernetes SIGs. *minikube Tutorials*. https://minikube.sigs.k8s.io/docs/tutorials/

[15] Go Authors. (2009–2024). *The Go Programming Language*. https://go.dev

[16] Aqua Security. (2020). *Kubernetes 101: Kubernetes Architecture*. https://www.aquasec.com/cloud-native-academy/kubernetes-101/kubernetes-architecture/

[17] DZero Labs. *Just-in-Time Kubernetes: A Beginner's Guide to Kubernetes Core Concepts*. Medium. https://medium.com/dzerolabs/just-in-time-kubernetes-a-beginners-guide-to-kubernetes-core-concepts-19ee7acbafa1

[18] LeapCell. *Learning Large-Scale Go Project Architecture from K8s*. Medium. https://leapcell.medium.com/learning-large-scale-go-project-architecture-from-k8s-6c8f2c3862d8

[19] Kim, J. (2022). *What Even Is A Mux?* Dev.to. https://dev.to/jpoly1219/what-even-is-a-mux-4fng

[20] Docker. *containerd vs Docker*. Docker Blog. https://www.docker.com/blog/containerd-vs-docker/

[21] Wikipedia. *Kubernetes*. https://en.wikipedia.org/wiki/Kubernetes

[22] Go Programming Tutorials. *Go Programming Language Tutorial*. YouTube Playlist. https://youtube.com/playlist?list=PLmD8u-IFdreyh6EUfevBcbiuCKzFk0EW_

[23] Finkel, R. A., & Bentley, J. L. (1974). Quad Trees: A Data Structure for Retrieval on Composite Keys. *Acta Informatica*, 4(1), 1–9. https://doi.org/10.1007/BF00288933

[24] Samet, H. (1990). *The Design and Analysis of Spatial Data Structures*. Addison-Wesley. ISBN: 978-0201502558

[25] Czaplicki, E. (2012–2024). *The Elm Architecture*. https://guide.elm-lang.org/architecture/

[26] Kubernetes Authors. (2024). *Kubernetes Controller Pattern*. https://kubernetes.io/docs/concepts/architecture/controller/

[27] Poitras, F. (2019–2024). *K9s: Kubernetes CLI To Manage Your Clusters In Style*. GitHub. https://github.com/derailed/k9s

[28] Jesseduffield. (2019–2024). *Lazydocker: A simple terminal UI for Docker and Docker Compose*. GitHub. https://github.com/jesseduffield/lazydocker

[29] Go Authors. (2024). *sync/atomic - The Go Standard Library*. https://pkg.go.dev/sync/atomic

[30] Go Authors. (2024). *runtime - Memory Statistics in Go*. https://pkg.go.dev/runtime#MemStats

[31] Go Authors. (2024). *encoding/json - The Go Standard Library*. https://pkg.go.dev/encoding/json

[32] Milton Bradley. (1967). *Battleship (game)*. Original pencil and paper game dates to World War I. https://en.wikipedia.org/wiki/Battleship_(game)